

Spring JPA Exercise 3: Transaction Rollback Testing

Objectives

Demonstrate writing Hibernate Query Language and Native Query

HQL stands for Hibernate Query Language, JPQL stands for Java Persistence Query Language, Compare HQL and JPQL, @Query annotation, HQL fetch keyword, aggregate functions in HQL, Native Query, nativeQuery attribute

Reference - <https://docs.jboss.org/hibernate/orm/4.3/devguide/en-US/html/ch11.html>

Features of JPA Query - <https://www.baeldung.com/spring-data-jpa-query>

Explain the need and benefit of Criteria Query

Scenarios where Criteria Query helps, CriteriaBuilder, Criteria Query, Root, TypedQuery

Reference - <https://docs.oracle.com/javaee/6/tutorial/doc/gjrij.html>

Hands on 1

Introduction to HQL and JPQL

HQL stands for Hibernate Query Language

JPQL stands for Java Persistence Query Language

Both HQL and JPQL are object focused query language similar to SQL

JPQL is a subset of HQL

All JPQL queries are valid HQL query, but the reverse is not true

Both HQL and JPQL allows SELECT, UPDATE and DELETE

HQL additionally allows INSERT statement

Reference - <https://docs.jboss.org/hibernate/orm/4.3/devguide/en-US/html/ch11.html>

Hands on 2

Get all permanent employees using HQL

Using HQL get all permanent employees. When retrieving the employee details it should also retrieve respective department and skill list as well.

HQL Solution

Include a new method definition in EmployeeRepository with @Query annotation

@Query(value="SELECT e FROM Employee e WHERE e.permanent = 1")

List<Employee> getAllPermanentEmployees();

// NOTE: HQL looks like SQL, instead of table, Java classes and it's

// instance variables are addressed here

Include appropriate service method

Include a new test method and Invoke the service method in OrmLearnApplication.java. Refer test method below that logs all employee details and each employee's skill details.

```
public static void testGetAllPermanentEmployees() {
```

Spring JPA Exercise 3: Transaction Rollback Testing

```
LOGGER.info("Start");
List<Employee> employees = employeeService.getAllPermanentEmployees();
LOGGER.debug("Permanent Employees:{", employees);
employees.forEach(e -> LOGGER.debug("Skills:{", e.getSkillList()));
LOGGER.info("End");
}
```

Check the list of SQL queries executed in the log file. Following queries would have got executed.

```
select employee0_.em_id as em_id1_2_0_, department1_.dp_id as dp_id1_1_1_,
skill3_.sk_id as sk_id1_4_2_, employee0_.em_date_of_birth as em_date_2_2_0_,
employee0_.em_dp_id as em_dp_id6_2_0_, employee0_.em_name as em_name3_2_0_,
employee0_.em_permanent as em_perma4_2_0_,
employee0_.em_salary as em_salar5_2_0_, department1_.dp_name as dp_name2_1_1_,
skill3_.sk_name as sk_name2_4_2_, skilllist2_.es_em_id as es_em_id1_3_0_,
skilllist2_.es_sk_id as es_sk_id2_3_0_
from   employee employee0_ left outer join department department1_ on
employee0_.em_dp_id=department1_.dp_id left outer join employee_skill skilllist2_ on
employee0_.em_id=skilllist2_.es_em_id left outer join skill skill3_ on
skilllist2_.es_sk_id=skill3_.sk_id
where  employee0_.em_permanent=1
select employeeeli0_.em_dp_id as em_dp_id6_2_0_, employeeeli0_.em_id as em_id1_2_0_,
employeeeli0_.em_id as em_id1_2_1_, employeeeli0_.em_date_of_birth as em_date_2_2_1_,
employeeeli0_.em_dp_id as em_dp_id6_2_1_, employeeeli0_.em_name as em_name3_2_1_,
employeeeli0_.em_permanent as em_perma4_2_1_, employeeeli0_.em_salary as em_salar5_2_1_
from   employee employeeeli0_ where employeeeli0_.em_dp_id=3
select employeeeli0_.em_dp_id as em_dp_id6_2_0_, employeeeli0_.em_id as em_id1_2_0_,
employeeeli0_.em_id as em_id1_2_1_, employeeeli0_.em_date_of_birth as em_date_2_2_1_,
employeeeli0_.em_dp_id as em_dp_id6_2_1_, employeeeli0_.em_name as em_name3_2_1_,
employeeeli0_.em_permanent as em_perma4_2_1_, employeeeli0_.em_salary as em_salar5_2_1_
from   employee employeeeli0_ where employeeeli0_.em_dp_id=2
select skilllist0_.es_em_id as es_em_id1_3_0_, skilllist0_.es_sk_id as es_sk_id2_3_0_,
skill1_.sk_id as sk_id1_4_1_, skill1_.sk_name as sk_name2_4_1_
from   employee_skill skilllist0_ inner join skill skill1_
on     skilllist0_.es_sk_id=skill1_.sk_id
where  skilllist0_.es_em_id=2
```

Optimizing HQL Solution by removing the EAGER fetch configuration

An optimal solution should not execute multiple queries, we have defined unnecessary fetch configuration in

Spring JPA Exercise 3: Transaction Rollback Testing

Eager fetch configuration is defined for employeeList in Department.java and skillList of Employee.java. Remove these two eager fetch configurations and check the logs. The following queries would have got executed. It would have failed when getting the skill list. Since we have removed the eager fetch, skill is not retrieved.

```
select employee0_.em_id as em_id1_2_, employee0_.em_date_of_birth as em_date_2_2_,
employee0_.em_dp_id as em_dp_id6_2_, employee0_.em_name as em_name3_2_,
employee0_.em_permanent as em_perma4_2_, employee0_.em_salary as em_salar5_2_
from   employee employee0_
where  employee0_.em_permanent=1

select department0_.dp_id as dp_id1_1_0_, department0_.dp_name as dp_name2_1_0_
from   department department0_
where  department0_.dp_id=2

select department0_.dp_id as dp_id1_1_0_, department0_.dp_name as dp_name2_1_0_
from   department department0_
where  department0_.dp_id=3
```

There are two issues in this approach:

We did not get the skill details

Still the query is not optimal as we have three queries executed

Optimizing HQL solution by using 'fetch'

Change the query in EmployeeRepository.java as specified below:

```
@Query(value="SELECT e FROM Employee e left join e.department d left join e.skillList WHERE
e.permanent = 1")
```

The above query still fails to get skill details. Include fetch after each join. Wherever data is required we can include fetch, which will populate the respective data. Change the query as specified below:

```
@Query(value="SELECT e FROM Employee e left join fetch e.department d left join fetch e.skillList WHERE
e.permanent = 1")
```

Following the single query generated for the above HQL:

```
select employee0_.em_id as em_id1_2_0_, department1_.dp_id as dp_id1_1_1_,
skill3_.sk_id as sk_id1_4_2_, employee0_.em_date_of_birth as em_date_2_2_0_,
employee0_.em_dp_id as em_dp_id6_2_0_, employee0_.em_name as em_name3_2_0_,
employee0_.em_permanent as em_perma4_2_0_, employee0_.em_salary as em_salar5_2_0_,
department1_.dp_name as dp_name2_1_1_, skill3_.sk_name as sk_name2_4_2_,
skilllist2_.es_em_id as es_em_id1_3_0_, skilllist2_.es_sk_id as es_sk_id2_3_0_
from   employee employee0_ left outer join department department1_ on
employee0_.em_dp_id=department1_.dp_id left outer join employee_skill skilllist2_ on
employee0_.em_id=skilllist2_.es_em_id left outer join skill skill3_ on
```

Spring JPA Exercise 3: Transaction Rollback Testing

skilllist2_.es_sk_id=skill3_.sk_id

where employee0_.em_permanent=1

IMPORTANT TAKEAWAY: Join keyword links the table, but does not populate the beans. Fetch ensures that the beans are populated. Based on our need wherever we need data, we can define fetch. When joining table data is not needed the fetch can be ignored.

Hands on 3

Fetch quiz attempt details using HQL

In a quiz application there is a requirement for admin to view details of a quiz that an user had attempted. This view should include the following details:

Username

Attempted Date

All questions as part of the attempt

List of options under each quiz

The option that is correct answer

The score for correct answer

Schema Diagram

Notes on Schema:

Tables user, question and options are self explanatory. They hold the respective master data.

Tables attempt, attempt_question and attempt_option are used to hold the data of attempts made by each user.

Follow steps below to setup the schema:

Go to spring-data-jpa-files folder in windows explorer

Open file quiz.mwb in MySQL Workbench

Generate SQL file using File > Export > Forward Engineer SQL CREATE Script

Click Browse and select the file name and folder for saving the generated SQL file

Select the check box "Generate INSERT Statements for Tables"

Click Next > Next > Finish to generate the SQL file

Execute the SQL file in ormlearn schema and check the data in the tables

Steps to get this implemented:

Create necessary entity class for each table defined above

Define necessary O/R mapping based on the schema defined above???

Create a Repository and Service class:

AttemptRepository

```
public Attempt getAttempt(int userId, int attemptId)
```

Spring JPA Exercise 3: Transaction Rollback Testing

AttemptService

```
public Attempt getAttempt(int userId, int attemptId)
```

Modify OrmLearnApplication.java t- include a new test method and test AttemptService.getAttemptDetail() method

Create HQL that joins the tables in the below order:

user

attempt

attempt_question

question

attempt_option

options

In the HQL include where clause for userId and attemptId

Include 'fetch' in HQL wherever there is one-to-many or many-to-many relationship

In OrmLearnApplication.java test method ??????get the attempts details, iterate through the details and display the data in the following format. The second column in each option denotes the score from question table. The last column in each option denotes the answer selected by the user.

What is the extension of the hyper text markup language file?

- 1) .xhtm 0.0 false
- 2) .ht 0.0 false
- 3) .html 1.0 true
- 4) .htmx 0.0 false

What is the maximum level of heading tag can be used in a HTML page?

- 1) 5 0.0 false
- 2) 3 0.0 true
- 3) 4 0.0 false
- 4) 6 1.0 false

The HTML document itself begins with <html> and ends </html>. State True or False

- 1) false 0.0 false
- 2) true 1.0 true

Choose the right option t- store text value value in a variable

- 1) 'John' 0.5 true
- 2) John 0.0 false
- 3) "John" 0.5 false
- 4) /John/ 0.0 false

Hands on 4

Get average salary using HQL

Spring JPA Exercise 3: Transaction Rollback Testing

Compute the average salary of a department using HQL.

Refer steps below to implement:

Define HQL in EmployeeRepository

```
@Query(value="SELECT AVG(e.salary) FROM Employee e")
```

```
double getAverageSalary();
```

Include new method with above signature in EmployeeService and include test method in OrmLearnApplication

The above query does not filter the result based on department id. Modify the query and method signature as specified below to accept department.

```
@Query(value="SELECT AVG(e.salary) FROM Employee e where e.department.id = :id")
```

```
double getAverageSalary(@Param("id") int id);
```

NOTES:

Observe how department id is referred from 'e'

Make note of the colon (:) used to define a parameter within a query

@Param annotation helps in binding the input department id with the query parameter

Similar to AVG(), all other aggregate functions can be used

Hands on 5

Get all employees using Native Query

About Native Queries

Native queries are direct SQL queries to the database instead of using HQL

Try to avoid Native Queries and make it minimal.

Avoiding native queries helps in easier portability of database

Follow steps below to implement

Define a new native query method in EmployeeRepository

```
@Query(value="SELECT * FROM employee", nativeQuery = true)
```

```
List<Employee> getAllEmployeesNative();
```

Define relevant method in service and OrmLearnApplication and test it

Hands on 6

Criteria Query

Find below an online retail user scenario

User goes to Amazon

Searches with keyword "laptop"

Spring JPA Exercise 3: Transaction Rollback Testing

The left hand side contains the following filter criteria categories:

Customer review

Hard Disk Size

RAM Size

CPU Speed

Operating System

Weight

CPU

The user might select options available in one or more of the criteria and try a fresh search.

In the above given scenario, what will be the where clause of the HQL query that you will run on the product?

The where clause varies based on the criteria selected by the user. We have to dynamically frame the where clause filters based on the criteria selected by user.

Criteria Query helps in handling this scenario in a better way. The filter criteria can be programmatically added, rather than fixing the HQL Statement.

Go through the examples in the link below to understand how Criteria Query has to be implemented.

<https://howtodoinjava.com/hibernate/hibernate-criteria-queries-tutorial/>